

Master's Thesis

Generating Efficient Optimization Formulations Using Large Language Models

Nathan Ferchtandiker

Student number: 13986163
Date: July 15, 2025
Master's program: Data Science and Business Analytics
Supervisor: Prof. Dick den Hertog
Second Reader: Britt van Veggel

Faculty of Economics and Business



UNIVERSITY OF AMSTERDAM
Economics & Business

Statement of Originality

This document is written by Student Nathan Ferchtandiker who declares to take full responsibility for the contents of this document. I declare that the text and the work presented in this document are original and that no sources other than those mentioned in the text and its references have been used in creating it. I have not used generative AI to generate or rewrite text. UvA Economics and Business is responsible solely for the supervision of completion of the work and submission, not for the contents.

Abstract

This thesis explores the use of Large Language Models (LLMs) to automate the generation of efficient reformulations for Mixed-Integer Optimization (MIO) problems. Traditionally, identifying effective formulations has relied heavily on human expertise and intuition, often involving a time-consuming, trial-and-error process without guarantees of improved computational performance. Such approaches can overlook alternative formulations that may yield significant speedups. To address this, the thesis proposes a novel pipeline that leverages LLMs to systematically generate, validate, and benchmark alternative optimization formulations. The pipeline consists of three key stages, namely (1) canonicalization of the original problem and input data, (2) automated generation of candidate reformulations by identifying structural inefficiencies and analyzing the problem context, and (3) validation using synthetic data generated by LLMs conditioned on the problem description. Results are benchmarked on large-scale optimization instances. A curated dataset of 10 optimization problems was constructed, each including a natural language problem descriptions, along with both efficient and inefficient formulations. Empirical results demonstrate that the pipeline can discover multiple alternative formulations per problem, with some achieving over 90% reductions in computational workload. The approach centers on identifying implicit assumptions in the original formulation and relaxing them when the problem context permits, thereby uncovering more efficient models. Additionally, an equivalence falsifier is introduced to assess whether two formulations are meaningfully different by comparing their optimal objective values across multiple instances.

Acknowledgments

I would like to express my gratitude to the individuals who have supported and guided me throughout this research journey. First, I would like to thank Professor Dick den Hertog, my thesis supervisor, for his supervision and expertise in constructing efficient optimization models. His guidance has been instrumental in shaping the direction and quality of this research, especially in determining the characteristics of efficient optimization modeling. I am particularly grateful for his role in enabling the research collaboration between our team, Professor Madeleine Udell from Stanford University, and Dr. Segev Wasserkrug from IBM Research. This collaboration has significantly enriched the scope and impact of this work. I extend my sincere appreciation to Professor Madeleine Udell from Stanford University for her participation in our research collaboration and for providing supervision throughout this project. Her expertise and the knowledge accumulated by her team during the development of OptiMUS (AhmadiTeshnizi et al., 2025) have been invaluable to this research. Additionally, I am grateful for the opportunity to visit Stanford for two months, which provided me with an intensive and enlightening view of how research is conducted at a world-class institution. Finally, I would like to thank Dr. Segev Wasserkrug from IBM Research for his valuable contributions of ideas and for coordinating the research collaboration. His insights and organizational efforts have been crucial in bringing together this research outcome. This thesis represents not only my individual work but also the collective effort and support of these exceptional researchers. Their mentorship, collaboration, and encouragement have made this research possible and have shaped my development as a researcher.

Contents

1	Introduction: The Art of Optimization Modeling	6
1.1	Optimization	6
1.2	Modeling Decisions Affecting Efficiency	6
1.2.1	Choice of Paradigm	6
1.2.2	Choice of Model Design	7
1.2.3	Choice of Primitives	7
1.2.4	Choice of Solver	8
1.3	Opportunities for LLMs	8
1.4	Contributions	9
2	Motivating Examples	10
3	Related Work	14
3.1	Language Models for Optimization Modeling	14
3.1.1	Datasets	14
3.1.2	Optimization Copilots	14
3.2	Large Language Models for Efficient Code Generation	14
3.3	Efficiency Enhancements for Mixed-Integer Optimization	15
4	Dataset	16
5	Methodology	18
5.1	Definitions	18
5.2	Pipeline	19
5.2.1	Stage 1: Initialization	19
5.2.2	Stage 2: Formulation Generation	21
5.2.3	Stage 3: Formulation Validation	22
5.3	Technical Implementation Details	24
5.4	Evaluation Metrics	25
6	Results	26
6.1	Reformulation Generation Performance	26
6.2	Performance Comparison	26
6.3	Invalidation Analysis	27
6.4	Qualitative Insights and Observations	28
6.5	Summary	28
7	Conclusion	29
8	Bibliography	30
A	Appendix	34
A.1	Performance Comparison Metrics	34
A.2	Prompts	34
A.2.1	Main Pipeline Prompts	34
A.2.2	Direct LLM Approach Prompts	37
A.2.3	Debugging and Error Correction Prompts	38

A.3	World Food Program Case Study	38
A.3.1	Problem Description	38
A.3.2	Dataset Structure	39
A.3.3	Inefficient Formulation	40
A.3.4	Efficient Formulation	41

1. Introduction: The Art of Optimization Modeling

1.1 Optimization

Optimization is a collection of mathematical techniques capable of finding an optimal solution to a problem, typically by maximizing or minimizing an objective function subject to some constraints. Optimization modeling is used in many business applications such as production scheduling, delivery routing, workforce scheduling, and more (Geem et al., 2001; Gurobi Optimization, LLC, 2024; Hillier and Lieberman, 2015). An optimization model typically consists of five components, namely sets, parameters, decision variables, objective function, and constraints. The decision variables need to be set such that the objective function is maximized or minimized while adhering to all constraints. The optimization model then typically takes the form of

$$\begin{array}{ll}
 \min & \phi(c, x) \quad \text{Objective function } \phi \text{ with cost coefficients } c, \\
 \text{subject to} & \psi_i(a_i, x) \geq b_i, \quad \forall i \in I, \quad \text{Inequality constraints } \psi_i \text{ parameterized by } a_i \text{ and bounded by } b_i \text{ over set } I, \\
 & \gamma_j(d_j, x) = e_j, \quad \forall j \in J, \quad \text{Equality constraints } \gamma_j \text{ parameterized by } d_j \text{ over set } J, \\
 & x_k \in \mathcal{D}_k, \quad \forall k \in K, \quad \text{Domain constraints where each variable } x_k \text{ lies in its domain } \mathcal{D}_k.
 \end{array}$$

Typically, an optimization expert formulates the optimization model and then uses optimization solver software to compute optimal values for the decision variables. The computational effort required to find a solution to an optimization model varies. Various properties of the optimization model affect the computational effort required to find an optimal solution or get arbitrarily close to it, as seen in Table 1.1. These and more factors need to be considered by the optimization expert when creating and solving the model. Usually, several formulations of a problem exist, which balance these aspects differently. For example, one formulation may be small in size but is weaker than a larger formulation. As a consequence, it is difficult to judge which formulation will be the most efficient prior to solving.

1.2 Modeling Decisions Affecting Efficiency

To solve an optimization model efficiently and balance the aspects mentioned in table 1.1, an optimization modeler typically takes several decisions during the modeling process. These decisions can be categorized into four major groups, which are typically made sequentially throughout the modeling process.

1.2.1 Choice of Paradigm

The first major decision is the choice of the optimization paradigm. This establishes the fundamental approach to solving the problem and heavily influences computational performance. Not all problems can be solved with every paradigm, but sometimes multiple paradigms are applicable in which case the expert needs to choose. In *Convex Optimization* (CO), optimization models consist of convex objective and inequality constraint functions, with the requirement that equality constraints must be affine. This structure ensures that the feasible region is convex. Adherence to these conditions ensures that any local optimum is globally optimal (Boyd and Vandenberghe, 2004). *Dynamic Optimization* (DO) problems have a special recursive structure of subproblems where solutions to smaller subproblems can be substituted into larger problems which can largely reduce redundant computations and search space (Bellman, 1966). *Heuristic Optimization* (HO) sacrifices guaranteed optimality for computational speed by using problem-specific approximations. Techniques include greedy methods, local search, simulated annealing, and

Table 1.1: **Mathematical Properties Affecting Optimization Efficiency.** Optimization model formulations make tradeoffs between these properties. Thus, it is difficult to judge, which formulation will be the most efficient prior to solving.

Property	Description	Reference
Conic Structure	A special convex model structure that is composed of affine functions and convex cones, which are vector spaces closed under positive multiplication. Conic structures can be exploited by special solvers.	Ben-Tal and Nemirovski, 2001
Convexity	The problem has a convex feasible region and a convex objective function, ensuring that any local optimum is a global optimum.	Boyd and Vandenberghe, 2004
Linearity	All model components are linear, which implies convexity and ensures continuity and differentiability. Linear optimization is structured and can be efficiently solved by algorithms that exploit these structures.	Bertsimas and Tsitsiklis, 1997
Model Size	Refers to the number of variables and constraints. Larger models typically require more computational resources and time due to increased memory requirements and operations.	Wright, Nocedal, et al., 1999
Sparsity	Measures the number of zero elements in the model. Sparse models are more efficient due to reduced memory usage and faster implementations of mathematical operations on sparse structures.	Gondzio, 2012
Symmetry	A symmetric feasible region may contain multiple optimal solutions. While solvers can sometimes exploit symmetry, it often increases computational effort unless symmetry-breaking constraints are added.	Liberti, 2012
Variable Domain	Decision variables may be continuous or discrete. Discrete domains introduce combinatorial complexity, making the problem non-convex and discontinuous. The strength of formulation depends on how closely it can be approximated by its continuous relaxations, where stronger formulations are more efficient.	Hillier and Lieberman, 2015

genetic algorithms (Geem et al., 2001). *Mixed-Integer Optimization (MIO)* allows for continuous and discrete decision variables. Discrete variables allow optimization to be applied to a larger set of problems but introduce combinatorial complexity that significantly extends search of the solution space (Hillier and Lieberman, 2015).

1.2.2 Choice of Model Design

Following paradigm choice, model design deals with the process of translating real-world problems into mathematical formulations. For CO, model design includes selecting the most appropriate standard form such as semi-definite models, where the model structure must adhere to an additional set of mathematical properties. For DO, model design focuses on identifying a suitable recursive structure and defining recursive equations. For HO, model design involves selecting or constructing heuristics that balance solution quality and computational speed. For MIO, design decisions include defining appropriate variables and their domains. This thesis focuses primarily on automating model design decisions for MIO, by using Large Language Models (LLMs) to identify alternative sets of decision variables to generate new formulations.

Model parameters are often based on real-world measurements. In such cases, the observed parameter values are stochastic, which can be accounted for in the model. The choice of how to model uncertainty in parameter values can significantly affect both the tractability and efficiency of the model. Techniques such as Stochastic Programming, Robust Optimization, and Chance-Constrained Programming offer different techniques for modeling uncertainty. However, this is an additional layer that can be applied to any of these paradigms and is not treated as a separate decision group within this framework but rather as a complementary consideration that modifies or extends the core modeling paradigm.

1.2.3 Choice of Primitives

The difference between the choice of primitives and the choice of model design is that modeling choices deal with the semantics of model components while primitive choices deal with the specific implementations of model components. After developing an initial model, primitives can be applied to improve computational performance. Primitives are mathematical transformations of the mathematical expressions composing the optimization model, which preserve

or approximate the original term. An example of a primitive is the substitution of the non-linear 1-norm in the objective function with an equivalent expression composed of linear expressions using auxiliary variables Sherali and Adams, 2013. Many solvers automatically detect expressions to which primitives can be applied (Cplex, 2009; Gurobi Optimization, LLC, 2024). Manually applying them can still lead to further computational gains as not all solvers can detect applicable model structures and apply primitives correctly.

1.2.4 Choice of Solver

The final group of decisions affecting optimization model efficiency involves the selection of a solver which internally determines the algorithms to use to find a solution. General-purpose solvers such as CPLEX (Cplex, 2009) and Gurobi (Gurobi Optimization, LLC, 2024) support a wide range of optimization model classes, while others are tailored for specific model structures such as MOSEK (ApS, 2025) for conic optimization and BARON (Sahinidis, 1996) for global non-convex optimization. Solvers differ significantly in the algorithms they implement, their ability to exploit model structure, and their computational efficiency for a given optimization model (Kronqvist et al., 2019).

In addition to selecting a solver, configuring its parameters can be critical. For MIO solvers, these settings control aspects such as the type of branching strategy used in the branch-and-bound algorithm, the presolve strategy, which is a phase in which the solver removes variables and constraints that are not relevant to the optimal solution, the aggressiveness of cutting plane generation, tolerance thresholds for optimality, and parallelism settings (Cplex, 2009; Gurobi Optimization, LLC, 2024). Fine-tuning these options can significantly reduce solution times, but may also introduce instability or inconsistency if misused.

Finally, to efficiently solve the optimization model, good software engineering practices should be applied, especially when handling large-scale instances. This ranges from choosing an efficient programming and modeling language to making use of efficient data structures and operations.

1.3 Opportunities for LLMs

Historically, automating decisions related to the choice of modeling paradigm and model design has been challenging, as these choices depend heavily on the specific context and semantics of the underlying problem. In contrast, decisions regarding the selection of mathematical primitives and solver algorithms are more amenable to automation, since they can often be guided by an analysis of the problem’s structural properties such as convexity, sparsity, or variable types. Recent advances in the development of LLMs have created new opportunities to enhance the efficiency of optimization models, especially at the stages of paradigm selection and model design. A key strength of LLMs lies in their ability to comprehend problem context directly from natural language descriptions. The effectiveness of LLMs in interpreting and solving optimization problem descriptions has been demonstrated by tools such as OptiMUS (AhmadiTeshnizi et al., 2025), ORLM (Huang et al., 2025), and LLMOPT (Jiang et al., 2025), which leverage LLMs to generate and solve optimization models from problem descriptions in natural language. For paradigm choice, LLMs can serve as an advisor in determining the applicable paradigm by suggesting optimization models that have solved similar problems for example. In modeling design choices, LLMs could assist in exploring alternative formulations, by proposing various ways of modeling the same problem and automatically testing formulations for validity and speed, similar to how this thesis explores the effectiveness of using LLMs to find different formulations for MIO problems. For the choice of primitives, modern solvers already apply many such transformations automatically by parsing the mathematical formulation. However, certain transformations become valid or effective when the broader problem context is taken into account. For instance, a complex non-linear term defined over a narrow input range could be approximated using a piecewise-linear function, significantly improving tractability without sacrificing much accuracy. In other cases, multiple primitive transformations may be applicable, each with different trade-offs. Here, LLMs can assist by evaluating the context and recommending the most suitable primitive based on both efficiency and fidelity to the original model. For the choice of solver, LLMs can serve as intelligent advisors by recommending solvers that align with the structural properties of a given model. They can also suggest relevant solver settings. Furthermore, LLMs can explain the implications of these settings, helping users make informed trade-offs between solution quality, speed, and robustness. As a result, LLMs have the potential to democratize advanced solver configuration, making high-performance optimization more accessible to non-experts while still offering valuable insights to practitioners. While current LLMs still require human oversight, their integration into the modeling workflow has the potential to reduce the time required to move from conceptual understanding to a solved model.

1.4 Contributions

Developing an efficient optimization model is a labor-intensive process, often requiring the exploration, implementation, and comparison of multiple modeling approaches and formulations. Yet, identifying the most efficient approach remains uncertain, as formulations involve complex trade-offs between competing factors such as size, structure, and solver compatibility. As optimization is increasingly deployed by large organizations to solve complex, large-scale problems, model efficiency becomes a critical concern. This thesis explores how LLMs can help automate model design decisions for MIO problems, reduce human effort, and improve modeling outcomes by evaluating their ability to support and enhance key modeling choices, particularly in the design of formulations for MIO. The main contributions are

- (1) **Identification of decision points** in the modeling process where LLMs enable new forms of automation in paradigm selection, model design, primitive application, and solver configuration.
- (2) **Proposal of a novel approach** for using LLMs to discover alternative formulations for MIO problems by systematically analyzing a given formulation in the context of a problem description.
- (3) **Development of a complete pipeline** that not only generates alternative formulations using LLMs but also identifies invalid formulations.
- (4) **Introduction of a synthetic data generation method** that produces problem data on demand for any arbitrary formulation, enabling automated empirical testing.
- (5) **Equivalence falsifier for optimization formulations** using multi-instance solving to detect mismatches in feasibility or optimal objective value.
- (6) **Creation of a benchmark dataset** of ten diverse optimization problems, each with a vague and precise natural language description, an inefficient formulation, and a known efficient counterpart.
- (7) **Publication of a Dataset** containing problem context, and both an efficient, and inefficient model formulation and large-scale problem data.

Experimental results show that the pipeline can discover alternative formulations that lead to solver workload reduction of over 90% on large-scale instances.

2. Motivating Examples

Every Optimization model contains implicit assumptions about its problem context. However, not all assumptions contained in the mathematical model need to hold when considering the context. Unnecessary implicit assumptions may increase the computational effort required to solve the model. Similarly, finding an efficient reformulation is oftentimes a challenge as they may be non-obvious, especially to inexperienced optimization modelers. To understand the concept of implicit assumptions and how identifying and relaxing them can yield more efficient formulations, consider the optimization problem below.

The goal is to choose up to p new hospital locations from a list of possible sites, in order to serve as many people living in nearby neighborhoods. A neighborhood is considered served if they are assigned to a hospital that is no more than S units away from their home. There are already M hospitals open, which must remain open.

An intuitive formulation for this problem is given by

$$\begin{aligned}
&\text{Sets: } I : \text{Households} \\
&\quad J_0 : \text{Existing hospitals} \\
&\quad J_1 : \text{Candidate hospital locations} \\
&\quad J = J_0 \cup J_1 : \text{All hospital sites} \\
&\text{Parameters: } v_i \in \mathbb{R}_+ : \text{People in household } i \in I \\
&\quad d_{ij} \in \mathbb{R}_+ : \text{Distance from } i \in I \text{ to } j \in J \\
&\quad a_{ij} \in \{0, 1\} : 1 \text{ if } d_{ij} \leq S, \text{ else } 0 \\
&\quad S \in \mathbb{R}_+ : \text{Maximum allowable travel distance} \\
&\quad p \in \mathbb{Z}_+ : \text{Maximum number of new hospitals to open} \\
&\text{Variables: } x_j \in \{0, 1\} : 1 \text{ if hospital } j \text{ is open, } 0 \text{ otherwise} \\
&\quad y_{ij} \in \{0, 1\} : 1 \text{ if household } i \text{ is assigned to hospital } j \\
&\text{Model: } \max \sum_{i \in I} \sum_{j \in J} v_i y_{ij} \quad \text{maximize number of people served} \\
&\text{s.t. } x_j = 1 \quad \forall j \in J_0 \quad \text{existing hospitals stay open} \\
&\quad \sum_{j \in J_1} x_j \leq p \quad \text{at most } p \text{ new hospitals opened} \\
&\quad \sum_{i \in I} y_{ij} \leq |I| x_j \quad \forall j \in J \quad \text{assign only to open hospitals} \\
&\quad \sum_{j \in J} y_{ij} \leq 1 \quad \forall i \in I \quad \text{each household assigned at most once} \\
&\quad y_{ij} \leq a_{ij} \quad \forall i \in I, \forall j \in J \quad \text{respect travel distance limit} \\
&\quad x_j \in \{0, 1\} \quad \forall j \in J \\
&\quad y_{ij} \in \{0, 1\} \quad \forall i \in I, \forall j \in J.
\end{aligned}$$

In this model, each household is individually assigned to a hospital. An implicit assumption in this formulation is that we care to know which hospital a household is assigned to, meaning the model assumes that it makes a difference if a household is assigned to hospital 1 or 2. This assumption is appropriate if, for example, hospitals have a limit on the number of households they can serve. However, since the model context does not explicitly state that a household can be covered by more than one hospital, this constitutes an unnecessary implicit assumption. Relaxing this assumption yields a more compact formulation using coverage variables indicating whether a household is covered by any hospital and is given by

$$\begin{aligned}
&\text{Sets: } I : \text{Households} \\
&\quad J_0 : \text{Existing hospitals} \\
&\quad J_1 : \text{Candidate hospital locations} \\
&\quad J = J_0 \cup J_1 : \text{All hospital sites} \\
&\text{Parameters: } v_i \in \mathbb{R}_+ : \text{Number of people in household } i \in I \\
&\quad d_{ij} \in \mathbb{R}_+ : \text{Distance from household } i \text{ to hospital } j \\
&\quad a_{ij} \in \{0, 1\} : 1 \text{ if } d_{ij} \leq S, 0 \text{ otherwise} \\
&\quad S \in \mathbb{R}_+ : \text{Maximum allowed travel distance} \\
&\quad p \in \mathbb{Z}_+ : \text{Maximum number of new hospitals to open} \\
&\text{Variables: } x_j \in \{0, 1\} : 1 \text{ if hospital } j \text{ is open, } 0 \text{ otherwise} \\
&\quad y_i \in \{0, 1\} : 1 \text{ if household } i \text{ is served within distance } S \\
&\text{Model: } \max \sum_{i \in I} v_i y_i \quad \text{maximize total population served} \\
&\text{s.t. } x_j = 1 \quad \forall j \in J_0 \quad \text{existing hospitals stay open} \\
&\quad \sum_{j \in J_1} x_j \leq p \quad \text{at most } p \text{ new hospitals can be opened} \\
&\quad y_i \leq \sum_{j \in J} a_{ij} x_j \quad \forall i \in I \quad \text{only count if covered by an open hospital} \\
&\quad x_j \in \{0, 1\} \quad \forall j \in J \\
&\quad y_i \in \{0, 1\} \quad \forall i \in I
\end{aligned}$$

It is not immediately obvious which formulation is more efficient. The first formulation is more sparse while the formulation with coverage variables contains fewer discrete variables. Figure 2.1 shows that the model with coverage variables is much faster. Now, consider the optimization problem described below.

A logistics company needs to transport pallets from a warehouse to a retail distribution center. There are different types of pallets, each containing a unique product category. Pallets of the same type are indistinguishable. For each pallet type, several identical pallets are available and need to be delivered. Each pallet occupies a certain amount of space in the truck and the space requirement differs between pallet types. Trucks used for transportation have a fixed capacity and can carry any combination of pallets as long as the total space used does not exceed this capacity. The goal is to determine how to load the pallets into the trucks such that the number of trucks is minimized while all pallets are transported and no truck exceeds its volume capacity.

Again, an intuitive formulation is given by

Sets: P : Pallets
 K : Candidate trucks

Parameters: $s_p \in \mathbb{R}_+$: Space occupied by pallet $p \in P$
 $C \in \mathbb{R}_+$: Capacity of each truck

Variables: $x_{pk} \in \{0, 1\}$: 1 if pallet p is loaded on truck k , 0 otherwise
 $y_k \in \{0, 1\}$: 1 if truck k is used, 0 otherwise

Model: $\min \sum_{k \in K} y_k$ minimize the number of trucks used

s.t. $\sum_{k \in K} x_{pk} = 1 \quad \forall p \in P$ each pallet is assigned to exactly one truck

$\sum_{p \in P} s_p x_{pk} \leq C y_k \quad \forall k \in K$ truck capacity constraint

$x_{pk} \in \{0, 1\} \quad \forall p \in P, \forall k \in K$

$y_k \in \{0, 1\} \quad \forall k \in K$

In this model, each pallet is individually assigned to a truck. An implicit assumption contained in this optimization model is that we distinguish between pallets of the same type, meaning that we care whether pallet 1 of type A goes to truck 1 or pallet 2 of type A goes into that truck. That is, even if two pallets are of the same product type and size, they are treated as distinct entities in the model. This assumption is appropriate if, for example, pallets are labeled individually (e.g. due to tracking, expiration dates, or inventory systems), or if the warehouse physically manages them as separate units. However, if all pallets of the same type are indistinguishable and interchangeable, this constitutes an unnecessary implicit assumption. Because the problem description states that pallets of the same type are indistinguishable, this assumption can be relaxed which yields a more compact formulation using aggregated variables given by

Sets: T : Pallet types
 K : Candidate trucks

Parameters: $n_t \in \mathbb{Z}_+$: Number of pallets of type $t \in T$
 $s_t \in \mathbb{R}_+$: Space occupied by one pallet of type t
 $C \in \mathbb{R}_+$: Capacity of each truck

Variables: $x_{tk} \in \mathbb{Z}_+$: Number of pallets of type t assigned to truck k
 $y_k \in \{0, 1\}$: 1 if truck k is used, 0 otherwise

Model: $\min \sum_{k \in K} y_k$ minimize number of trucks used

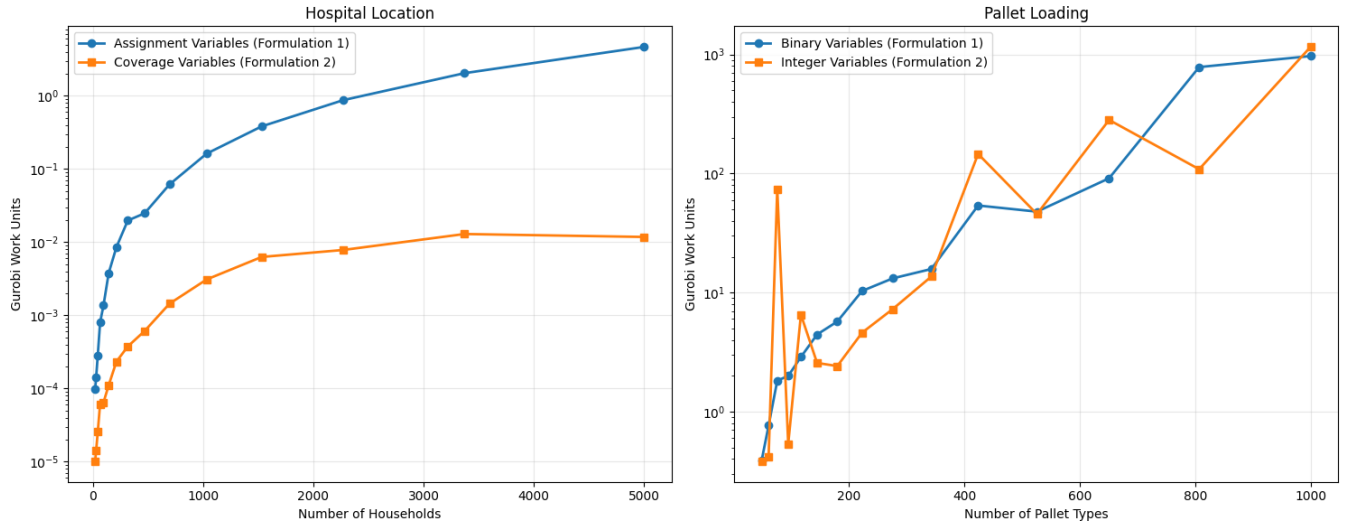
s.t. $\sum_{k \in K} x_{tk} = n_t \quad \forall t \in T$ all pallets must be shipped

$\sum_{t \in T} s_t x_{tk} \leq C y_k \quad \forall k \in K$ truck capacity constraint

$x_{tk} \in \mathbb{Z}_+ \quad \forall t \in T, \forall k \in K$

$y_k \in \{0, 1\} \quad \forall k \in K$

Figure 2.1: **Speed Comparison of Formulations.** The figure shows the number of work units for different problem sizes for all formulations. The work units are a deterministic and consistent measure for workload returned by the Gurobi solver (Gurobi Optimization, LLC, 2024). For the hospital location problem, the ratio of the number of candidate locations to the number of households is approximately 1:10. For the pallet loading case, there are between 1 and 5 pallets per type.



This example illustrates how identifying and challenging implicit assumptions in an optimization model offers guidance in identifying alternative formulations. In addition, Figure 2.1 shows the differences in workload required to solve each formulation at various scales. For the hospital location problem, reducing model size and symmetry in exchange for sparsity by using coverage variables had a positive effect for all problem sizes. On the other hand, speedups for the pallet loading problem are less consistent. The formulation using integer variables also reduces model size and symmetry in exchange for less sparsity. However, this formulation only performs consistently better for problems involving roughly 200 to 300 pallet types. This demonstrates that alternative formulations can lead to significant performance improvements, although it is not always clear beforehand which formulation will be superior in a given case. Therefore, to accurately assess the performance of a formulation, empirical testing is essential.

3. Related Work

3.1 Language Models for Optimization Modeling

3.1.1 Datasets

Performing optimization modeling with LLMs requires a dataset containing text descriptions, optimization models, and code to solve the model. Several such datasets exist, but usually only cover easy optimization problems. NL4Opt is a benchmarking competition for natural language processing systems for optimization modeling (Ramamonjison et al., 2022). They published a dataset containing 1101 easy linear optimization problem descriptions and annotated model representations. AhmadiTeshnizi et al. (2025) introduce a dataset containing 355 MIO problems with problem descriptions in natural language with associated optimization model in L^AT_EX, Python code, solution, and optimal objective value for problems in varying domains and of varying difficulty. Huang et al. (2025) created a synthetic dataset of 32,481 problem descriptions and expected text output from a language model containing explanations in natural language, a mathematical model in L^AT_EX, and code to solve the model. They synthesized the dataset by prompting an LLM to generate hypothetical scenarios in which optimization can be used and augment previously generated optimization models by changing the objective and constraints.

3.1.2 Optimization Copilots

Optimization copilots are LLM-based systems serving the purpose of supporting optimization practitioners in the development and solving of optimization models by automating parts of the optimization modeling process. Typically, these systems are given an optimization problem description in natural language as input and are tasked to solve it. OptiMUS (AhmadiTeshnizi et al., 2025) and Chain-of-Experts (Xiao et al., 2023) autonomously do so by first generating a mathematical model and subsequently converting it into code by leveraging different prompts at each stage. OptiMUS debugs code in case the execution yields an error. In addition, LLMOPT (Jiang et al., 2025) and ORLM (Huang et al., 2025) effectively show that fine-tuning LLMs on pairs of problem descriptions, mathematical models and code enhances performance. Opposed to other optimization copilots, Astorga et al. (2024) generate several candidate formulations by modifying generated formulations using Monte Carlo Tree Search. Candidate formulations are evaluated on correctness using LLMs and are compared on equivalence using symbolic pruning.

Other applications of LLMs in optimization modeling are the utilization of LLMs for equivalence checking (Zhai et al., 2025). They take a set of combinatorial optimization problems and modify them in a way that either preserves or removes equivalence. Next, they ask an LLM to create mappings between the decision variables of the original and modified optimization model. OptiChat (Chen et al., 2024) is a diagnostic debugging tool for infeasible optimization models. It uses LLMs to describe optimization models, identify sources of infeasibility, and suggest adjustments to make the model feasible.

3.2 Large Language Models for Efficient Code Generation

KernelBench (Ouyang et al., 2025) is a benchmarking framework designed to evaluate LLM-generated implementations of neural network kernels. This framework assesses both execution speed measured via wall-clock time and functional correctness, using a reference implementation as a baseline. Similarly, CodePDE (Li et al., 2025) introduces a framework that leverages LLMs to develop algorithms for solving partial differential equations (PDEs). Their system includes an iterative code-debugging loop. The LLM-generated algorithms were shown to outperform human-written algorithms on 4 out of 5 benchmark tasks. A notable advantage highlighted in their work is that LLM-generated code is efficient while remaining interpretable, unlike opaque neural computational engines.

Similarly, LLM-guided code implementation of optimization models enables performance improvements through modeling choices that are under the modeler’s control, rather than depending on opaque solver settings. These works demonstrate the potential of LLMs to autonomously generate efficient and human-readable code for mathematical and algorithmic problems, aligning closely with the goal of using LLMs to discover more efficient formulations of optimization models.

3.3 Efficiency Enhancements for Mixed-Integer Optimization

MIO models are highly expressive and widely used across industries, but their computational complexity often presents a major bottleneck, especially for large instances. A variety of strategies have been developed to improve the efficiency of solving MIO problems, targeting different aspects of the modeling process. One potential avenue for improving efficiency is to reformulate the problem under a different optimization paradigm, such as dynamic programming or heuristic methods. In certain instances, this shift has led to dramatic speedups, sometimes several orders of magnitude faster than the original MIO formulation (Brotcorne et al., 2013; Fischetti et al., 2018; Vanderbeck and Wolsey, 2010). However, such paradigm shifts are only applicable under specific structural conditions. For example, dynamic optimization requires a recursive problem structure, which limits its applicability to problems where such decomposition is possible. Within the same MIO paradigm, modifying the model design is one of the most common and impactful techniques for enhancing computational performance. A typical approach is to redefine the decision variables or restructure constraints to yield a more efficient formulation (Audet et al., 2004; Bertsimas and Patterson, 1998; Ramazan and Dimitrakopoulos, 2004). Despite its effectiveness, there is no general framework or methodology for systematically generating alternative formulations from an existing one. As a result, this process remains largely dependent on expert intuition, trial-and-error, and labor-intensive manual work. Mathematical primitives have also been studied extensively (Asghari et al., 2022; Sherali and Adams, 2013) and are implemented in most modern solvers (Cplex, 2009; Gurobi Optimization, LLC, 2024; Sahinidis, 1996). However, the sheer number of possible primitives, combined with the lack of transparency about which transformations are applied by a given solver, makes it difficult for modelers to determine which primitives should be manually applied to improve performance. Finally, the choice of solver itself can significantly impact performance. Solvers differ in their internal algorithms and their ability to exploit specific problem structures, often resulting in large variations in solution time across different solvers for the same problem instance (Anand et al., 2017). Additionally, solvers offer a wide range of configuration options that must often be manually tuned by the user (Cplex, 2009; Gurobi Optimization, LLC, 2024; Sahinidis, 1996). This tuning process is non-trivial and can be opaque, making it difficult to predict which solver and configuration will yield the best performance for a given model.

These challenges highlight the need for tools and frameworks that can assist in navigating the complex landscape of MIO modeling decisions. In particular, there is a gap in methods that can support the systematic discovery of more efficient model designs for a given MIO formulation, an area this thesis aims to address.

4. Dataset

Identifying efficient reformulations requires access to a data set that includes both problem context and paired examples of inefficient and efficient optimization model formulations. However, a key challenge is that most books and academic papers tend to present only the efficient, finalized formulations. This contrasts with real-world scenarios, where especially novice optimization modelers often begin with inefficient models. Another obstacle is the lack of accessible and standardized data for sets and parameters. Changing the formulation may imply that sets or parameters must be modified or added. Thus, the problem data needs to contain several consistent variants of sets and parameters required by multiple formulations.

To address this gap, part of this thesis is the publication of a dataset containing 10 Mixed-Integer Linear Optimization problems. Each problem comes with

- a detailed problem description that contains all necessary information for deriving a complete and valid optimization model,
- an inefficient mathematical model, written in $\text{\LaTeX}$ accompanied by code in GurobiPy (Gurobi Optimization, LLC, 2024), that represents a plausible but suboptimal formulation of the problem,
- an efficient mathematical model, also written in $\text{\LaTeX}$ and accompanied by code in GurobiPy, that corrects inefficiencies and improves computational performance and
- a large-scale dataset that includes all sets and parameters required by both formulations in a consistent manner.

This dataset currently includes 10 distinct optimization problems, each designed to showcase meaningful reformulations and highlight the trade-offs in model design.

Table 4.1: **Overview of Optimization Problems in Dataset.** An overview of optimization problems contained in the dataset with descriptions of inefficiencies and efficiency improvements.

Problem Name	Description	Inefficient Model	Efficient Model
Air Traffic Management	Scheduling of start and landing times of airplanes.	Tracks the departure status of airplanes over time with discrete variables.	Tracks the departure status of airplanes over time with continuous variables.
Blood Bank Locations	Facility location problem for blood storage.	Binary indicators for allocations between hospitals and distribution centers.	Binary indicators per hospital, indicating coverage status.
Food Distribution*	Food supplies must be bought and transported along a network.	Decision variables for food transported along paths.	Decision variables for food transported along arcs.
Hospital Placement	Placement of hospitals to maximize the number of citizens covered.	Binary indicators between neighborhoods and hospital locations.	Binary indicators per neighborhood indicating whether it is covered by any hospital.
Design of Residential Tower*	Selection of floor configurations for residential tower to maximize profits.	Binary indicators for combinations of design and floor.	Integer variable indicating the number of designs; floor allocation is done in post-processing.
Mobility Hub Locations*	Facility location problem for last-mile transportation providers.	Binary indicators for each path-hub pair.	Binary indicator per path showing whether it is covered by any hub.

Problem Name	Description	Inefficient Model	Efficient Model
Pit Mining	Mining blocks are scheduled over time to maximize mineral value.	Binary variables indicating whether a block is harvested in a time period.	Blocks with maximal value are modeled with continuous domains (fully mined).
Tax System	A tax system is designed to minimize deviation in expected taxes per taxpayer.	Tax rates and brackets are optimized together, leading to bilinear terms.	A large set of candidate brackets is fixed; only bracket-specific rates are optimized.
Disaster Facility Location	Logistics hubs must be placed to ensure full regional coverage at minimal cost.	Redundant assignment variables between regions and hubs.	Redundant variables and inefficient constraints are removed.
Workforce Scheduling in a Production Setting	Workers are assigned to machines to manufacture products requiring specific machine hours.	Specific workers are assigned to specific machines at specific times.	Integer variables indicate total hours worked per machine.

* *Development set.*

The data was synthesized using dedicated Python scripts designed to generate context-consistent problem data for each problem instance. These scripts programmatically construct sets and parameters by randomly sampling values within user-defined ranges. To ensure the integrity of the generated data, the code was carefully reviewed to account for dependencies between data points during generation. The motivation behind this approach is to produce data that accurately reflects the logical relationships, constraints, and structural dependencies inherent in the optimization formulations under study. A key requirement is the preservation of consistency between sets and parameters across different formulations. For instance, one formulation might represent a problem using nodes and edges in a network, while another may use precomputed paths. In such cases, it is essential that the underlying data remains consistent to ensure a fair comparison. Generating data programmatically allows precise control over these dependencies which can not always be guaranteed when relying on preexisting datasets.

Table 4.2: **Dataset Statistics.** Average and Standard Deviation Before Presolve. High standard deviations reflect the wide variability in problem instance sizes within the dataset. Some instances can be solved efficiently, which is compensated for by larger sizes.

Statistic	Average	Standard Deviation
Problem Data Size (MB)	95.07	140.26
# Variables (Inefficient Model)	2.06×10^6	3.20×10^6
# Constraints (Inefficient Model)	2.72×10^6	6.16×10^6
# Variables (Efficient Model)	2.75×10^5	7.65×10^5
# Constraints (Efficient Model)	1.56×10^4	2.22×10^4

5. Methodology

This work proposes a pipeline for leveraging LLMs to generate mathematically equivalent but computationally more efficient formulations of optimization problems. The pipeline is composed of three main stages, namely **initialization**, **formulation generation**, and **formulation validation**. The primary goal of the pipeline is to identify and remove unnecessary assumptions that degrade computational performance, while preserving the semantics of the original model through rigorous validation procedures.

5.1 Definitions

Problem data refers to a concrete dataset instance that supplies numerical values for all sets and parameters required to instantiate and solve a specific optimization model. Multiple distinct data instances can be associated with the same underlying problem, as long as they cover all sets and parameters used in this problem. This enables the evaluation of a single model under various scenarios and scales. The pipeline distinguishes between two categories of problem data. The *large problem data* refers to the large-scale problem data provided by the dataset described in chapter 4 and is used to benchmark the computational performance of the final selected formulation. It represents the target application scale. Meanwhile, *synthetic problem data* is a collection of smaller, synthetic problem data instances that are generated within the pipeline to evaluate the executability, validity, and average workload of candidate formulations. Despite the fact that the large problem data was also generated synthetically, it is not considered to be a part of the synthetic problem data for the sake of this pipeline. The *schema* of the problem data is defined as the set of column names and column data types, and is provided as input to the pipeline along with the large problem data. One challenge is ensuring that synthetic problem data instances are aligned to the schema of the large-scale problem data, meaning they contain data points for all keys provided in the large problem data and ensuring that the values are of the expected data type. To ensure that the synthetic data is aligned, the synthetic data sets are checked against the schema of the large problem data. While all instances associated with a problem share the same structural schema, they differ in the actual values assigned to sets and parameters.

In this work, a **canonical formulation** represents a fully specified mathematical and computational representation of an optimization model formulation. Each formulation defines a particular way to express the same problem and includes several components. The canonical formulation contains a *mathematical model* written in $\text{\LaTeX}$ consisting of sets, parameters, decision variables, constraints, and an objective function, providing a standardized and interpretable definition of the optimization problem. For every set and parameter in the canonical formulation, the LLM assigns a *data coverage flag*, which is a binary indicator specifying whether that set or parameter is present in the input scheme. If a formulation has data coverage for all its required sets and parameters, it is considered compatible with the given problem data. In addition, the canonical formulation includes *executable code* that is composed of a Python function that, when provided with a compatible problem data instance, programmatically constructs and returns a GurobiPy model object (Gurobi Optimization, LLC, 2024). This approach ensures that the mathematical structure and computational implementation are tightly coupled, as any valid problem data instance can be passed to the canonical formulation’s code to automatically instantiate a GurobiPy model. Canonical formulations are the primary objects manipulated by the pipeline. The pipeline generates, validates, and compares multiple formulations of the same problem, in order to discover variants that improve computational efficiency without altering the semantics or feasibility space of the original model.

Assumptions and recommendations are the central part in discovering and generating new formulations. In this work, an *assumption* refers to a modeling decision that alters the solution space of the optimization problem. These assumptions are generated by an LLM through analysis of the initial formulation and its problem context. Each assumption is characterized by four components. The components are *name*, *description*, *reasoning* behind the assumption’s inclusion in the original formulation, and a *breakability flag*, which is a binary indicator specifying whether the assumption is essential or if it can be safely relaxed or removed when considering the problem context.

The purpose of this step is to identify assumptions that are modeled but not strictly required to preserve the intended semantics of the problem.

For each identified assumption, the LLM is prompted to generate a corresponding *recommendation*, which outlines how the original formulation could be modified if that assumption were relaxed or removed. A recommendation provides a concrete strategy for altering the original formulation to eliminate the assumption while preserving the problem’s semantics. Each recommendation contains a *name*, *description*, the *assumption name* that this recommendation is referencing, the *suggested change* to the original formulation, and *reasoning* for the suggested change. Recommendations serve as intermediate representations between modeling assumptions and model formulations. They guide the LLM in producing new candidate formulations by explicitly linking assumptions to actionable modifications. This structured approach ensures that formulations are traceable, explainable, and targeted, focusing computational improvements on assumptions that are unnecessary or overly restrictive.

The **synthetic data generator** is a key component of the pipeline, enabling controlled and efficient evaluation of optimization formulations. Its main function is to produce random, synthetic problem data that strictly adheres to the schema of the large problem data. The pipeline relies on these smaller, synthetically generated instances to validate correctness, measure performance, and assess equivalence across different formulations.

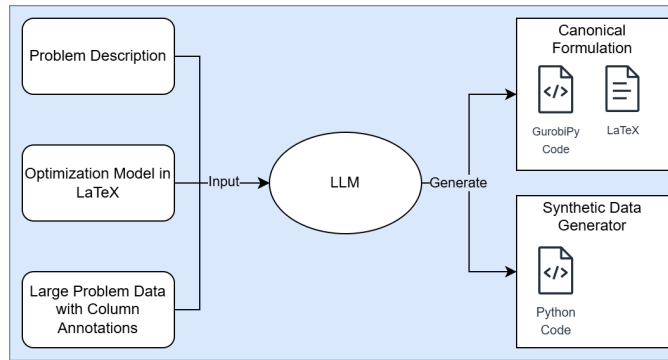
5.2 Pipeline

This work proposes a three-stage pipeline for leveraging LLMs to generate alternative formulations of optimization problems that are mathematically equivalent to the original but are potentially more efficient. The pipeline consists of a stage for **initialization**, **formulation generation**, and **formulation validation**.

5.2.1 Stage 1: Initialization

The purpose of the initialization stage is to make the pipeline compatible with a wide variety of inputs from the real world. In practice, optimization problems are often described in inconsistent or informal ways, ranging from natural language to mathematical formulations to code. To overcome this, the pipeline begins by normalizing and structuring this information into a canonical formulation to ensure that it can be processed procedurally.

Figure 5.1: **Formulation Canonicalization and Data Generator Creation.** An LLM processes a problem description, an optimization model, and the annotations of the large problem data to format the original formulation into a canonical formulation and create the synthetic data generator.



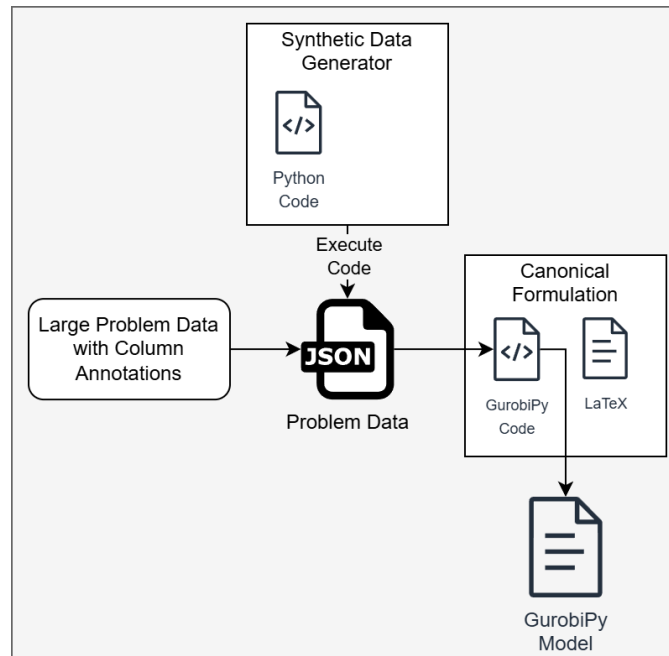
While the canonical component adds some robustness to the input format, the pipeline still requires problem context, a description of the original model, and a dataset with an annotated schema as input. In the context of this work,

- a description of the optimization problem in natural language,
- a LaTeX representation of the initial mathematical model and
- a large-scale problem data instance in JSON format with column annotations, where each column annotation includes its *name*, *description*, and *shape* fields, each specified in natural language

are provided as input to the pipeline (see Appendix A.3). Given these inputs, the LLM performs two key initialization tasks. The first step of the initialization is (1) *formulation canonicalization*. As mentioned above, the LLM standardizes the initial formulation in LaTeX and translates it into GurobiPy code which is structurally aligned with the annotated dataset schema and can be executed to solve instances of the optimization problem. Following that is the creation of the (2) *synthetic data generator*. The LLM is fed the problem context and data schema of the large-scale problem data based on which it generates a synthetic data generator to use Python code for producing synthetic problem data of a smaller scale. These synthetic instances conform to the schema of the provided dataset and are used in later stages to evaluate formulations efficiently and at scale.

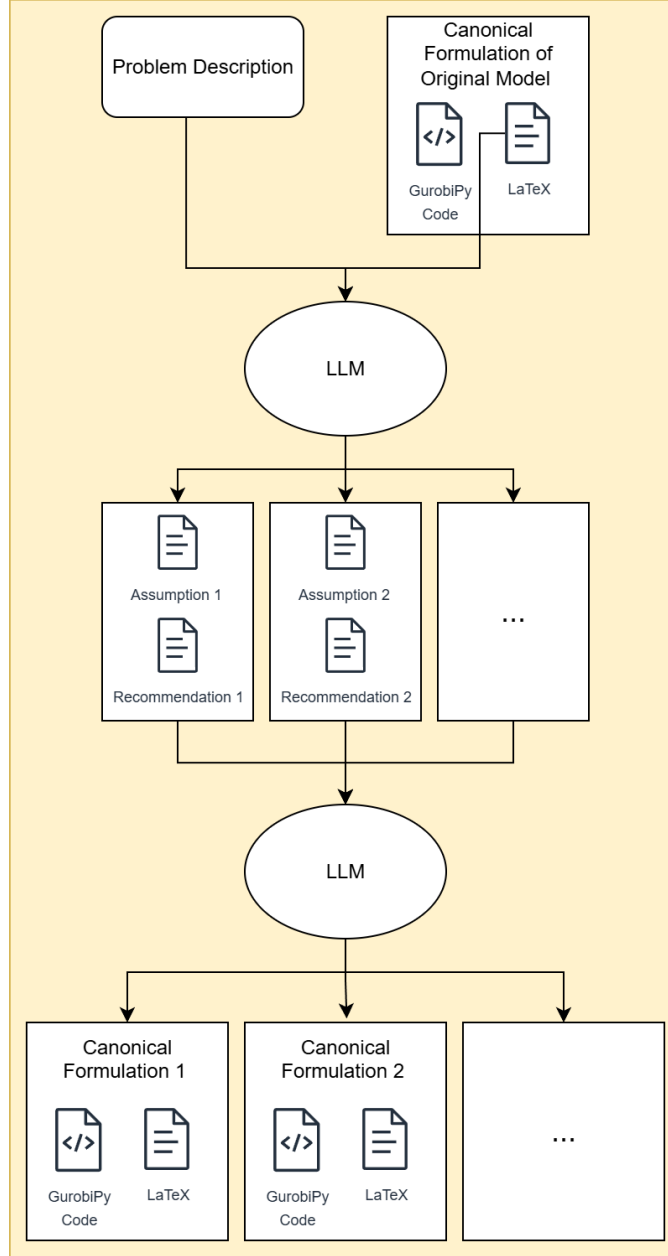
Once generated, the synthetic data generator code must be tested rigorously to ensure correctness and usability. Thus, the code of the synthetic data generator is subject to a debugging process. LLMs are prone to producing code that may include syntax and runtime errors (Sun et al., 2024). Therefore, the first step is the debugging of syntax and runtime errors ensuring that the generated Python code executes without errors, such as type mismatches, undefined variables, or other common runtime issues. Second, as mentioned above, it is important that the synthetic data matches the large problem data in terms of coverage and data format to ensure compatibility with the formulations and their GurobiPy implementations. Thus, the keys and associated data types of the values are validated against the schema of the large problem data to ensure that they match. Third, to ensure that the code generates feasible instances, a minimal fraction of datasets must be feasible when solved on the original formulation. Finally, during development, the data generator would oftentimes generate problem data instances that are too small to show notable performance differences or too large to initiate GurobiPy models in a reasonable time. Thus, bounds on the solve time of GurobiPy models initialized using synthetic problem data and the original formulation ensure that the datasets generated by the data generator are not too small nor too large. If the synthetic data generator fails any of these tests, an error is thrown containing the reason and error-dependent context. This error is propagated to the LLM, which then revises the implementation to correct the issue. If the debugging process fails after n attempts, the pipeline terminates without generating new formulations. Although there is no explicit validation for this aspect, the LLM is also instructed to maintain logical consistency between datasets. For example, if one formulation requires arc costs and another requires path costs, the values should be consistent across both representations.

Figure 5.2: **GurobiPy Model Initialization.** A GurobiPy model is initiated by passing a problem data instance to the canonical formulation which passes the data to its GurobiPy code to generate the GurobiPy instance. This ensures that a formulation can be run on all problem data instances.



5.2.2 Stage 2: Formulation Generation

Figure 5.3: **Formulation Generation.** A Pair of assumption and recommendation is generated by passing the problem context and optimization model in $\text{\LaTeX}$ to an LLM. Next, the LLM produces new formulations in canonical form.



The goal of the formulation generation stage is to produce alternative formulations of the original optimization model that may be computationally more efficient. This stage consists of two main steps. In the (1) *assumption extraction* step, the LLM systematically analyzes the problem context and the canonicalized model in $\text{\LaTeX}$ of the original formulation to identify implicit modeling assumptions. The analysis focuses on examining the mathematical structure, variable definitions, constraint formulations, and objective function to identify assumptions that may be unnecessarily restrictive. For each identified assumption, the LLM evaluates whether it can be safely relaxed or removed while preserving the intended semantics of the problem, and generates a corresponding recommendation outlining specific modifications to the original formulation. In the (2) *formulation synthesis* step, the LLM constructs a new formulation for each assumption–recommendation pair by modifying the original model accordingly. The LLM

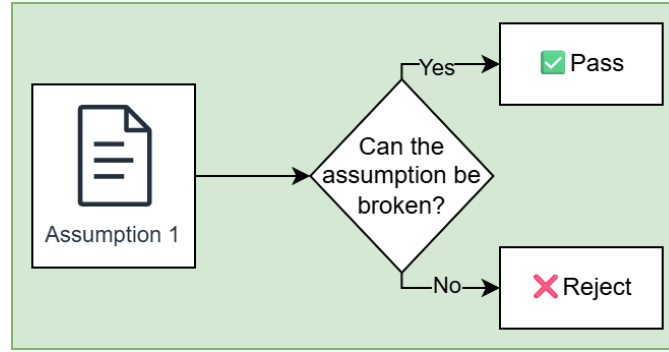
is instructed to leave the semantics of the objective function unchanged, as equivalence between the original and reformulated models will later be evaluated in Stage 3 by comparing their optimal objective values. The output of this stage is a set of candidate formulations, each representing a different modeling variant of the same underlying optimization problem. These candidates are passed to the next stage for validation and performance comparison.

5.2.3 Stage 3: Formulation Validation

Each candidate formulation generated in Stage 2 undergoes a validation process to ensure correctness and test equivalence with the original model. Validation is performed using several synthetic problem data instances produced by the synthetic data generator and consists of four sequential checks.

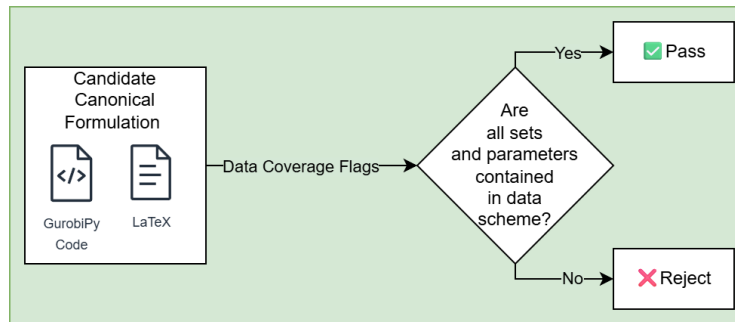
- (1) The *assumption breakability check* verifies whether the assumption used to derive the formulation can indeed be violated within the problem context, based on the assumption’s breakability flag. This check ensures that only meaningful reformulations are considered, filtering out trivial or invalid modifications.

Figure 5.4: **Assumption Breakability Check.** Each assumption has a boolean value indicating whether this assumption can be broken in the context of the problem. This boolean is set by LLM during the *assumption extraction* step of the second stage of the pipeline.



- (2) Next, the *data coverage check* ensures that all sets and parameters referenced in the formulation are present in the dataset schema. The formulation passes this check only if the data coverage flag is true for all components used. The data coverage flag is set by the LLM in the *formulation synthesis* step of the second pipeline stage. This check prevents formulations that require data not available in the problem context.

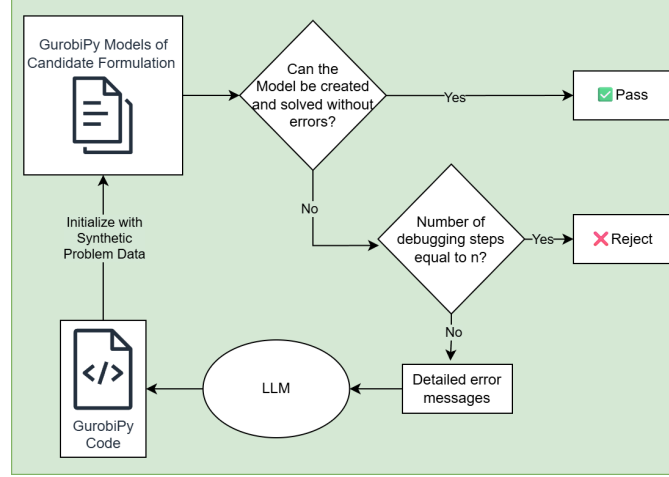
Figure 5.5: **Data Coverage Check.** Each set and parameter of a canonical formulation has a boolean value associated with it indicating if it is present in the scheme of the large problem data. If all sets and parameters are present in the data scheme, the formulation passes this check.



- (3) The *code executability check* confirms that the generated GurobiPy code runs without errors across multiple synthetic datasets. If errors occur, a debugging loop is triggered wherein a detailed error message is returned to

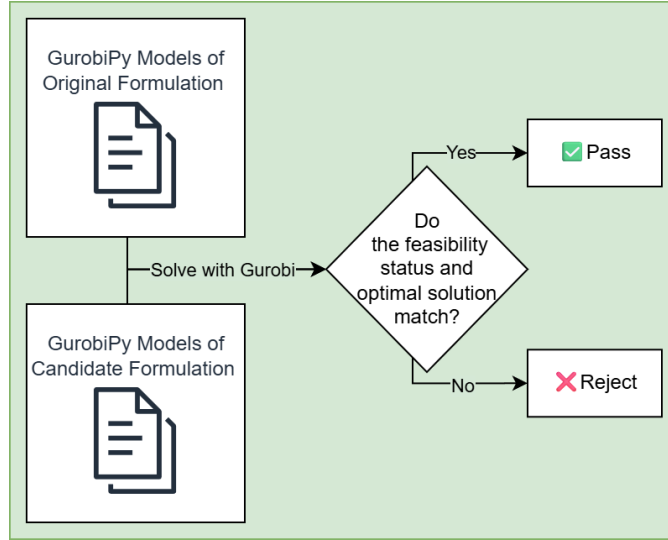
the LLM, which then attempts to refine the code and model structure. If a valid and executable optimization model is not produced after n refinement iterations, the formulation is marked as non-executable. This check addresses concerns about the reliability of automatically generated code by ensuring robust error handling and iterative refinement.

Figure 5.6: **Code Executability Check.** GurobiPy Models are initialized according to Figure 5.2 and solved with Gurobi. In case of syntax or runtime errors, the LLM is tasked to debug the GurobiPy code of the canonical formulation. The code executability check fails if the GurobiPy Model can not be executed after n debugging iterations.



- (4) Finally, the *objective value check* indicates whether a candidate formulation yields the same feasibility status and objective value as the original model across all synthetic problem data instances. This serves as a practical proxy for equivalence, but it is important to emphasize that this check is not an equivalence verifier, as it does not prove equivalence. Instead, it acts as an equivalence falsifier. If a candidate formulation produces a different objective value or feasibility status on any instance, it is guaranteed to be inequivalent to the original. However, matching objective values and feasibility statuses on a finite set of instances is only a necessary, not sufficient, condition for equivalence. True semantic equivalence would require formal proof or exhaustive verification across all possible instances, which is outside the scope of this thesis. Thus, the objective value check can only falsify, not verify, equivalence. EquivaMap (Zhai et al., 2025) offers an equivalence verifier by leveraging LLMs to generate variable mappings. However, the reformulations studied in this thesis often require dynamic, algorithmic relationships between variables, for example, reconstructing arc flows from path-based formulations. Therefore, EquivaMap failed to create correct mappings between the inefficient and the efficient formulations described in chapter 4 during the development of the pipeline.

Figure 5.7: **Objective Value Check.** The GurobiPy model of the original formulation and candidate formulation are initialized on several synthetic problem data instances generated by the synthetic data generator and solved with Gurobi. The check is an equivalence falsifier: if any instance yields a different objective value or feasibility status, the candidate is deemed inequivalent.



Formulations that pass all four checks are deemed valid. Among the valid candidate formulations, the one with the lowest average work units during solving on the synthetic datasets is selected as the *fastest* formulation. This formulation is then benchmarked on the full-scale real-world dataset to compare performance against the original model in terms of both model generation and solve time. However, as discussed in section 2, the fact that a formulation is the fastest on small synthetic datasets does not guarantee that it will also be the fastest on the large-scale problem data. This is because the synthetic datasets may not accurately capture performance-relevant mathematical properties of the large problem data such as scale, symmetry, or sparsity.

5.3 Technical Implementation Details

The pipeline employs OpenAI’s GPT-4.1 model with prompts, that were selected based on an initial testing in which several prompting approaches were evaluated qualitatively to ensure consistent and reliable output. In the assumption extraction phase, the LLM is provided with explicit examples of common optimization modeling assumptions and their potential relaxations. These include variable aggregation for cases where individual decisions can be derived through post-processing, removal of fixed variables that do not contribute to the optimization, elimination of redundant variables through strict equality constraints, integer relaxation when the problem context guarantees optimality preservation, and alternative problem structures that maintain semantic equivalence. More details on the prompts are provided in Appendix A.2.

A central element in ensuring the pipeline’s robustness and reliability is the use of Pydantic models for structured data validation and type checking. Pydantic (Colvin et al., 2025) is a Python library that enables users to define data structures as Python classes with explicit type annotations and field descriptions in natural language. These classes can be instantiated from strings that follow a format specified by Pydantic. Moreover, Pydantic can automatically generate natural language instructions for the required string format based on the class definitions. These instructions are included in the prompts of the pipeline. Once the LLM generates an output, Pydantic validates that the output matches the expected format and parses it into Python objects. In this pipeline, Pydantic models are created for canonical formulation, assumption, recommendation, and the synthetic data generator. This means that every time the LLM generates output, the output is parsed and validated against the corresponding Pydantic model before it is used further in the pipeline. The use of Pydantic provides several critical benefits for the pipeline’s reliability. Type safety ensures that all data structures are validated at runtime, preventing type-related errors that could propagate through the pipeline and cause failures in later stages. Format checks by Pydantic ensure that LLM

outputs conform to expected formats, reducing the likelihood of malformed or inconsistent results that could lead to invalid formulations.

The pipeline incorporates robust runtime-error handling mechanisms for LLM-generated code to address concerns about the reliability of the code. When errors occur during GurobiPy model instantiation or synthetic data generation, the system captures stack traces and provides a debugging prompt to the LLM, which then attempts to refine the code. This process is repeated up to a specified maximum number of attempts for each code snippet. This systematic approach ensures that the pipeline can recover from common errors such as type mismatches, undefined variables, and constraint formulation issues, making it more robust than naive code generation approaches.

To address skepticism about the reliability of performance comparisons, the pipeline uses Gurobi’s deterministic *Work* metric rather than wall-clock time. The Work metric represents the deterministic amount of solver effort spent during optimization, providing a consistent measure of computational complexity. This choice eliminates concerns about system load and multi-threading that could affect performance measurements. The pipeline also implements statistical validation by testing formulations on multiple synthetic datasets and computing average performance metrics.

5.4 Evaluation Metrics

The effectiveness of the formulation pipeline is assessed using a comprehensive set of quantitative metrics that capture both the quality and practicality of the generated formulations. These metrics are designed to provide objective evidence of the pipeline’s effectiveness and address potential skepticism about its reliability.

Number of Formulations Generated (N_{reform}). The total number of distinct candidate formulations generated is

$$N_{\text{reform}} = |\mathcal{F}|, \quad (5.1)$$

where $\mathcal{F}$ is the set of all formulations proposed by the LLM. This metric provides insight into the LLM’s ability to identify multiple potential improvements for a given optimization problem.

Count of Valid Formulations (n_{valid}). The total number of generated formulations that pass all validation checks is

$$n_{\text{valid}} = |\mathcal{F}_{\text{valid}}|, \quad (5.2)$$

where $\mathcal{F}_{\text{valid}} \subseteq \mathcal{F}$ is the set of valid formulations. This metric provides a direct measure of how many generated formulations meet the required quality standards, addressing concerns about LLM reliability.

Work Units on Large Problem Data (W_f). To quantify the computational effort required by each model, we use Gurobi’s *Work* metric, which represents the deterministic amount of solver effort spent during optimization. Unlike wall-clock time, this measure is consistent across runs on the same hardware and settings. A work unit roughly corresponds to one second of single-core wall-clock CPU time. The total work units required to solve a given model f is given by

$$W_f, \quad (5.3)$$

where W_f denotes the number of Gurobi work units needed to solve formulation f . This metric captures the total solver workload associated with a model’s structure, independent of external system factors.

Invalidation Counts by Check (n_{fail}^i). To identify common failure points in the generation pipeline, we count the number of formulations that fail each validation as

$$n_{\text{fail}}^i = |\mathcal{F}_{\text{fail}}^i|, \quad (5.4)$$

where $\mathcal{F}_{\text{fail}}^i \subseteq \mathcal{F}$ is the set of formulations that fail validation check $i \in \{\text{assumption breakability, data coverage, code executability, objective value match}\}$. This metric highlights where the generation process most often fails, providing actionable insight into weaknesses in the LLM-generated formulations and guiding targeted improvements to increase robustness.

These metrics collectively provide comprehensive evidence of the pipeline’s effectiveness, addressing concerns about LLM reliability, formulation quality, and performance improvement validation. The use of deterministic metrics and multiple validation layers ensures that the results are robust and reproducible.

6. Results

This chapter presents the empirical evaluation of the proposed pipeline for generating optimization model reformulations. The goal is to assess the pipeline’s ability to generate valid and performant formulations compared to direct, naive inputs to an LLM. We report results in terms of reformulation validity, performance gains, and failure modes.

6.1 Reformulation Generation Performance

The pipeline was able to generate an average of 7.4 formulations per problem (see Table 6.1). Of those, approximately 49% were deemed valid after undergoing a series of tests. Importantly, the generated formulations often showed variation in performance on synthetic datasets, suggesting that they were meaningfully distinct from each other and not redundant variants.

Despite the overall success rate, the pipeline struggled with certain problems. Specifically, it failed to produce any valid formulations for problems *P2* and *P3*. For *P2*, four of the six generated formulations failed due to insufficient data coverage, while the remaining two were deemed inequivalent. In the case of *P3*, all seven generated formulations were inequivalent to the original, except for one which failed due to a code execution issue. These results demonstrate that while the pipeline is capable of exploring a diverse solution space, it is still susceptible to failure modes depending on the structure and complexity of the problem.

Table 6.1: **Reformulation Generation Summary.** The pipeline produces several new formulations for each problem 5.1. 49% of generated formulations passed all validation checks. The pipeline produced at least one valid formulation for 8 out of 10 problems.

Problem	Generated Formulations (5.1)	Valid Formulations (5.2)	(Percent)
P1	10	9	(90%)
P2	6	0	(0%)
P3	7	0	(0%)
P4	5	4	(80%)
P5	10	4	(40%)
P6	6	2	(33%)
P7	8	7	(88%)
P8	10	3	(30%)
P9	7	5	(71%)
P10	5	2	(40%)
Average	7.4	3.6	(49%)

6.2 Performance Comparison

To evaluate performance, we compared the solving workload of the original formulations with those produced by the pipeline and by direct (naive) LLM inputs. Table 6.2 summarizes these comparisons. In multiple cases, both the pipeline and the naive LLM inputs achieved dramatic speedups with some exceeding 90%. However, the pipeline consistently produced faster formulations across a wider range of problems. Nevertheless, some caveats exist. For instance, in problem *P5*, the pipeline produced a slow formulation that had to be terminated before completion, similar to the original formulation. The observed negative performance change suggests that the pipeline-generated formulation consumed more computational resources. However, given that both formulations had to be terminated due to time limits, the formulations generated by the pipeline actually consumed more workload in the same time

frame, indicating better parallelizability. Indeed, the pipeline formulation achieved a terminal optimality gap of 0.43%, compared to 0.50% for the original formulation, indicating slightly better convergence behavior under identical time constraints. Such cases highlight the importance of not only measuring execution time but also examining solver behavior and resource utilization when evaluating reformulations.

Table 6.2: **Performance Improvement in Work Units.** Comparison of solving workload on large problem data (5.3) between the original formulation, the fastest valid model generated by the pipeline, and naive LLM Input. See Appendix A.1 for an equivalent analysis on the wall-clock time.

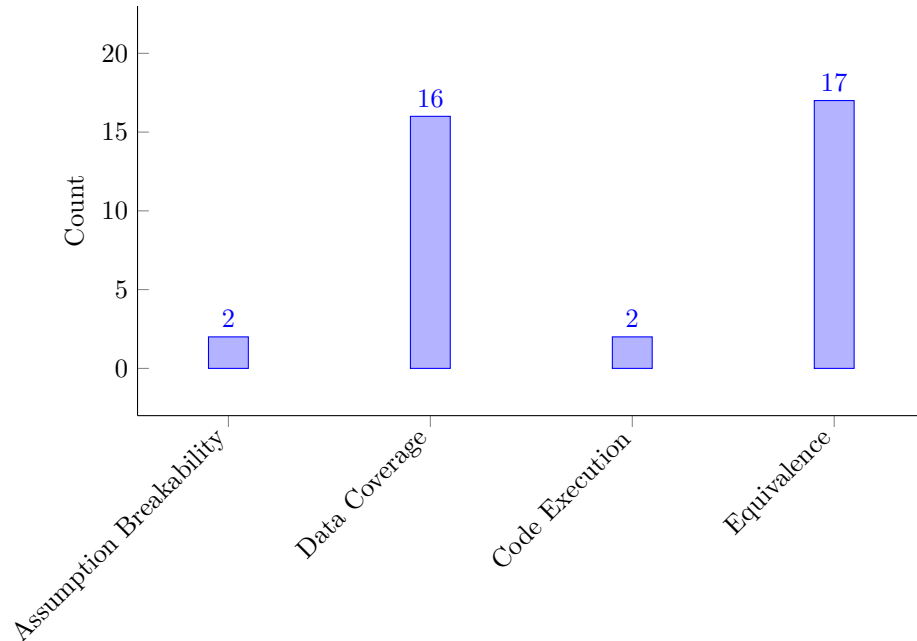
Problem	Original	Reformulation of Pipeline	Naive Input to LLM
P1	29.31	0.79 (97.31%)	-
P2	429.72	-	35.77 (91.6%)
P3	*2790.01	-	-
P4	26.38	1.64 (93.80%)	1.64 (93.80%)
P5	*1735.17	*2378.03 (-37.05%)	-
P6	247.42	19.49 (92.12%)	-
P7	*2156.19	3.87 (99.8%)	4.69 (99.78%)
P8	131.37	0.59 (99.55%)	-
P9	12.88	3.05 (76.34%)	5.17 (59.85%)
P10	4.36	0.19 (95.59%)	-

* Indicates a forced termination before completion after 60 minutes.

6.3 Invalidation Analysis

Figure 6.1 provides a breakdown of the most common reasons for reformulation invalidation. The majority of invalid cases were attributed to either a lack of data coverage or mathematical inequivalence. Specifically, 16 formulations failed due to inadequate data coverage, and 17 were rejected for not being mathematically equivalent to the original. Failures due to code execution errors or assumption violations were relatively rare. This pattern highlights the importance of rigorous semantic checks and a robust data generator. Encouragingly, debugging efforts on the data generator proved effective, as all datasets generated after the debugging phase yielded feasible optimization models.

Figure 6.1: **Reasons Why Formulations Are Invalid.** This histogram shows the count for reasons why formulations were flagged as invalid 5.4



6.4 Qualitative Insights and Observations

Beyond quantitative metrics, several qualitative insights emerged from the evaluation of the pipeline. Manual inspection revealed that out of the eight fastest formulations identified, only six were actually equivalent to the original formulation, proving that the objective value check used in the pipeline does not guarantee equivalence. On a more positive note, the debugging procedures implemented for the pipeline’s code generation step proved largely effective. Of the 56 generated formulations that passed assumption and data coverage checks, only two exhibited coding errors. While the number of debugging iterations of the data generator was not systematically tracked, improvements were sometimes made after more than 10 iterations, solving issues related to runtime, feasibility, or excitability.

6.5 Summary

The results confirm that the proposed pipeline can generate a diverse and often valid set of reformulations, with meaningful performance improvements over naive LLM prompting. However, the occasional generation of invalid or no equivalent formulations illustrates the need for improved semantic validation, and equivalence checking. Debugging and validation components, especially for data generation and code synthesis, proved critical to achieving the observed reliability and performance gains.

7. Conclusion

This thesis investigates the potential of LLMs to automatically generate alternative formulations for mathematical optimization problems, with a focus on improving computational efficiency. A structured pipeline was proposed, incorporating assumption identification, synthetic data generation, validation and steps to guide the reformulation process. The framework introduced in this work identifies and explicitly presents implicit assumptions to the LLM and has been proven to be a helpful mechanism for guiding the model toward several alternative formulations. Comparing the alternative formulations that the naive LLM input produced, it was shown that the pipeline adds robustness to the formulations because the pipeline found valid formulations for more cases. The results demonstrate that LLMs are indeed capable of generating more efficient reformulations in several cases. In particular, the pipeline was able to discover formulations that significantly outperformed the original models in terms of solving time, sometimes by over 90% in required workload. However, not all reformulations were valid or equivalent, and the fastest solutions were not always semantically faithful to the original formulation.

Despite these promising results, several limitations must be acknowledged. First, the evaluation was conducted on a relatively small benchmark dataset consisting of only 10 optimization problems, which limits the generalization of the findings. In some cases, the transformations required to achieve faster formulations were quite similar, such as in *P2* and *P4*. Second, the mechanism used to verify equivalence relies on comparing optimal objective values across a set of synthetic datasets. However, this approach does not guarantee true mathematical equivalence, underscoring the need for human oversight in the pipeline. Moreover, the pipeline selects its fastest formulation based on computational expense of solving the synthetic datasets. However, the generated synthetic datasets may not accurately reflect important mathematical properties such as sparsity or symmetry of large-scale datasets. In addition, the scale of the synthetic datasets is intentionally smaller compared to the large problem data. Chapter 2 shows that formulations may scale differently with problem size, so selecting a formulation based on its performance on small synthetic datasets may not yield the most efficient formulation for the actual problem due to poor scaling.

Ultimately, the pipeline should be viewed as a tool for generating and exploring promising modeling ideas rather than a fully automated end-to-end solution for finding more efficient formulations. Its strength lies in its ability to suggest alternative formulations that may not be immediately obvious to a human modeler. By automating parts of the reformulation and evaluation process, the pipeline can accelerate the discovery of potentially more efficient formulations. However, selecting the best formulation for a real-world problem still requires human judgment, particularly when considering factors such as mathematical equivalence, problem-specific structure, and scalability. In this sense, the pipeline serves as a valuable aid in the modeling process, supporting but not replacing expert insight.

Future work focusing on synthetic data generation for optimization models should put particular attention to enabling user control over key aspects like dataset size, data generation procedure, data diversity and allow replication of mathematical properties. Additionally, it is crucial to ensure that generated instances are feasible with respect to the original problem constraints. While the debugging loop used in the pipeline adjusted the data generator to produce a feasible dataset, the synthetic datasets lack a feasibility guarantee. Another important direction for future work is to improve the strength of the equivalence checking mechanism. One possible strategy is to design datasets in such a way that each constraint in the model is binding for at least one instance in the dataset. Another approach could involve starting from an optimal solution and gradually perturbing problem parameters until that solution is no longer optimal in the original formulation. The same perturbation could then be applied to the candidate formulation to observe whether its optimal solution also changes in the same way. If both formulations respond consistently, it would provide stronger evidence of behavioral equivalence. This would help ensure that all parts of the formulation are meaningfully exercised during evaluation, increasing confidence in the equivalence check.

Overall, this work provides a step toward harnessing LLMs for automated modeling tasks and highlights both the potential and challenges of AI-assisted optimization model design.

8. Bibliography

- AhmadiTeshnizi, A., Gao, W., Brunborg, H., Talaei, S., Lawless, C., & Udell, M. (2025). Optimus-0.3: Using large language models to model and solve optimization problems at scale. <https://arxiv.org/abs/2407.19633>
- Anand, R., Aggarwal, D., & Kumar, V. (2017). A comparative analysis of optimization solvers. *Journal of Statistics and Management Systems*, 20(4), 623–635.
- ApS, M. (2025). *The mosek python fusion api manual. version 11.0*. <https://docs.mosek.com/latest/pythonfusion/index.html>
- Asghari, M., Fathollahi-Fard, A. M., Mirzapour Al-e-hashem, S. M. J., & Dulebenets, M. A. (2022). Transformation and linearization techniques in optimization: A state-of-the-art survey. *Mathematics*, 10(2). <https://doi.org/10.3390/math10020283>
- Astorga, N., Liu, T., Xiao, Y., & van der Schaar, M. (2024). Autoformulation of mathematical optimization models using llms. *arXiv preprint arXiv:2411.01679*.
- Audet, C., Brimberg, J., Hansen, P., Digabel, S. L., & Mladenović, N. (2004). Pooling problem: Alternate formulations and solution methods. *Management Science*, 50(6), 761–776.
- Bellman, R. (1966). Dynamic programming. *Science*, 153(3731), 34–37.
- Ben-Tal, A., & Nemirovski, A. (2001). *Lectures on modern convex optimization: Analysis, algorithms, and engineering applications*. SIAM.
- Bertsimas, D., & Patterson, S. S. (1998). The air traffic flow management problem with enroute capacities. *Operations research*, 46(3), 406–422.
- Bertsimas, D., & Tsitsiklis, J. N. (1997). *Introduction to linear optimization* (Vol. 6). Athena scientific Belmont, MA.
- Boyd, S. P., & Vandenberghe, L. (2004). *Convex optimization*. Cambridge university press.
- Brotcorne, L., Hanafi, S., & Mansi, R. (2013). One-level reformulation of the bilevel knapsack problem using dynamic programming. *Discrete Optimization*, 10(1), 1–10.
- Chen, H., Constante-Flores, G. E., & Li, C. (2024). Diagnosing infeasible optimization problems using large language models. *INFOR: Information Systems and Operational Research*, 62(4), 573–587.
- Colvin, S., Jolibois, E., Ramezani, H., Garcia Badaracco, A., Dorsey, T., Montague, D., Matveenko, S., Trylesinski, M., Runkle, S., Hewitt, D., Hall, A., & Plot, V. (2025, June). *Pydantic* (Version v2.11.7). <https://github.com/pydantic/pydantic>
- Cplex, I. I. (2009). V12. 1: User’s manual for cplex. *International Business Machines Corporation*, 46(53), 157.
- Fischetti, M., Monaci, M., & Sinnl, M. (2018). A dynamic reformulation heuristic for generalized interdiction problems. *European Journal of Operational Research*, 267(1), 40–51.
- Geem, Z. W., Kim, J. H., & Loganathan, G. (2001). A new heuristic optimization algorithm: Harmony search. *SIMULATION*, 76(2), 60–68.
- Gondzio, J. (2012). Interior point methods 25 years later. *European Journal of Operational Research*, 218(3), 587–601.
- Gurobi Optimization, LLC. (2024). Gurobi Optimizer Reference Manual. <https://www.gurobi.com>
- Hillier, F. S., & Lieberman, G. J. (2015). *Introduction to operations research*. McGraw-Hill.
- Huang, C., Tang, Z., Hu, S., Jiang, R., Zheng, X., Ge, D., Wang, B., & Wang, Z. (2025). Orlm: A customizable framework in training large models for automated optimization modeling. <https://arxiv.org/abs/2405.17743>
- Jiang, C., Shu, X., Qian, H., Lu, X., Zhou, J., Zhou, A., & Yu, Y. (2025). Llmopt: Learning to define and solve general optimization problems from scratch. <https://arxiv.org/abs/2410.13213>
- Kronqvist, J., Bernal, D. E., Lundell, A., & Grossmann, I. E. (2019). A review and comparison of solvers for convex minlp. *Optimization and Engineering*, 20, 397–455.
- Li, S., Marwah, T., Shen, J., Sun, W., Risteski, A., Yang, Y., & Talwalkar, A. (2025). Codepde: An inference framework for llm-driven pde solver generation. *arXiv preprint arXiv:2505.08783*.
- Liberti, L. (2012). Symmetry in mathematical programming. *Mixed Integer Nonlinear Programming*, 263–283.

- Ouyang, A., Guo, S., Arora, S., Zhang, A. L., Hu, W., Ré, C., & Mirhoseini, A. (2025). Kernelbench: Can llms write efficient gpu kernels? *arXiv preprint arXiv:2502.10517*.
- Ramamonjison, R., Yu, T., Li, R., Li, H., Carenini, G., Ghaddar, B., He, S., Mostajabdaveh, M., Banitalebi-Dehkordi, A., Zhou, Z., & Zhang, Y. (2022). Nl4opt competition: Formulating optimization problems based on their natural language descriptions. In M. Ciccone, G. Stolovitzky, & J. Albrecht (Eds.), *Proceedings of the neurips 2022 competitions track* (pp. 189–203, Vol. 220). PMLR.
- Ramazan, S., & Dimitrakopoulos, R. (2004). Recent applications of operations research and efficient mip formulations in open pit mining. *SME Transactions*, 316.
- Sahinidis, N. V. (1996). Baron: A general purpose global optimization software package. *Journal of Global Optimization*, 8, 201–205.
- Sherali, H. D., & Adams, W. P. (2013). Reformulation–linearization techniques for discrete optimization problems. In P. M. Pardalos, D.-Z. Du, & R. L. Graham (Eds.), *Handbook of combinatorial optimization* (pp. 2849–2896). Springer New York.
- Sun, Z., Zhu, H., Xu, B., Du, X., Li, L., & Lo, D. (2024). Llm as runtime error handler: A promising pathway to adaptive self-healing of software systems. <https://arxiv.org/abs/2408.01055>
- Vanderbeck, F., & Wolsey, L. A. (2010). Reformulation and decomposition of integer programs. In M. Jünger, T. M. Liebling, D. Naddef, G. L. Nemhauser, W. R. Pulleyblank, G. Reinelt, G. Rinaldi, & L. A. Wolsey (Eds.), *50 years of integer programming 1958-2008: From the early years to the state-of-the-art* (pp. 431–502). Springer Berlin Heidelberg.
- Wright, S., Nocedal, J., et al. (1999). Numerical optimization. *Springer Science*, 35(67-68), 7.
- Xiao, Z., Zhang, D., Wu, Y., Xu, L., Wang, Y. J., Han, X., Fu, X., Zhong, T., Zeng, J., Song, M., et al. (2023). Chain-of-experts: When llms meet complex operations research problems. *The Twelfth International Conference on Learning representations*.
- Zhai, H., Lawless, C., Vitercik, E., & Leqi, L. (2025). Equivamap: Leveraging llms for automatic equivalence checking of optimization formulations. *arXiv preprint arXiv:2502.14760*.

List of Figures

2.1	Speed Comparison of Formulations	13
5.1	Formulation Canonicalization and Data Generator Creation	19
5.2	GurobiPy Model Initialization	20
5.3	Formulation Generation	21
5.4	Assumption Breakability Check	22
5.5	Data Coverage Check	22
5.6	Code Executability Check	23
5.7	Objective Value Check	24
6.1	Reasons Why Formulations Are Invalid	27

List of Tables

1.1	Mathematical Properties Affecting Optimization Efficiency	7
4.1	Overview of Optimization Problems in Dataset	16
4.2	Dataset Statistics	17
6.1	Reformulation Generation Summary	26
6.2	Performance Improvement in Work Units	27
A.1	Performance Improvement in Wall-Clock Time	34

A. Appendix

A.1 Performance Comparison Metrics

This section provides detailed definitions and measurements for the performance comparison between the original and reformulated optimization models.

Solve Time on Large Problem Data (T). To quantify the computational effort required by each model, we use Gurobi’s reported *Solve Time* in wall-clock seconds. Solve time directly reflects the end-to-end latency a user observes. The total solve time required to solve a given model f is

$$T_f, \tag{A.1}$$

where T_f denotes the number of seconds Gurobi spends to reach optimality (or terminate) for formulation f . As in Table 6.2, the fastest formulation was selected based on the formulation with the lowest workload on the synthetic datasets.

Table A.1: **Performance Improvement in Wall-Clock Time.** Comparison of solving time on large problem data (A.1) between the original formulation, the fastest valid model generated by the pipeline, and naive LLM Input.

Problem	Original	Reformulation of Pipeline	Naive Input to LLM
P1	97.65	27.18 (72.16%)	-
P2	1243.91	-	82.58 (93.36%)
P3	*3600.87	-	-
P4	63.66	2.56 (95.98%)	2.65 (95.83%)
P5	3600.11	3600.07	(0.00%)
P6	1112.26	120.73 (89.14%)	-
P7	*3601.87	3.81 (99.89%)	4.69 (99.87%)
P8	1541.41	3.93 (99.75%)	-
P9	24.99	4.71 (81.14%)	6.70 (73.18%)
P10	10.98	0.43 (96.04%)	-

* Indicates a forced termination before completion after 60 minutes.

A.2 Prompts

This section provides a comprehensive list of the prompts used throughout the research for generating and optimizing mathematical optimization models.

A.2.1 Main Pipeline Prompts

Formulation Generation Prompt

The following prompt is used to generate complete optimization formulations from problem descriptions:

You are an expert in mathematical optimization and GurobiPy modeling.
Generate a formulation for the given optimization problem.

Key requirements:

1. Create a clear and descriptive name for the formulation.
2. Provide a detailed description of the formulation.
3. Define all necessary variables with appropriate types and descriptions.
4. Define all constraints with clear descriptions and expressions.
5. Ensure all code is well-structured, efficient, and includes comments.
6. Use the provided data explanation to infer set and parameter references in the dataset.
7. The dataset is a JSON dictionary; access data using bracket notation only, without default values.
8. Always use the exact keys from the data explanation when accessing data.
9. For dictionary parameters, use nested dictionary access for multi-indexed parameters (e.g., `data['param']`).
10. For single-indexed parameters, use string keys.
11. Never hardcode values or use keys that do not exist in the data.

Here is the description of the dataset

The dataset is a Python dictionary where all keys are strings. It contains all sets and parameters required

- Sets are represented as lists of strings, even if the key is an integer.
- Parameters are represented as either single values, dictionaries (for indexed parameters), or nested dictionaries.
- All data is accessible using bracket notation, e.g., `data["set_name"]`, `data["param_name"]`, or `data["param_name"]`.
- The structure and required fields of the dataset are determined by the model's data description.

Here is the description of the code to generate the GurobiPy model

- Use the GurobiPy Python API (`import gurobipy as gp, from gurobipy import GRB`).
- Define all sets, parameters, variables, constraints, and the objective function as described in the model.
- Access all data using the provided Python dictionary (`data`), using bracket notation (e.g., `data["set_name"]`).
- Use clear and descriptive variable and constraint names.
- Add comments explaining each part of the model.
- Ensure the model is complete and can be solved with `model.optimize()`.
- Do not include any code for data input/output or reading from files.
- The code should be a single Python function that takes a data dictionary as input and returns a GurobiPy

Example function signature:

```
def generate_gurobipy_model(data: dict) -> "gurobipy.Model":  
    # your code here
```

Please ensure the formulation is complete and implementable in GurobiPy.

The JSON must follow this schema:

```
{pydantic_format_instructions}
```

Generate a formulation based on the following model context:

```
{user_input_model_context}
```

Here is the data description:

```
{user_input_data_structure}
```

Please ensure the formulation is complete and implementable in GurobiPy.

Data Generator Prompt

Used to generate synthetic datasets for optimization problems:

You are an expert in Python programming and data generation for optimization problems. Your task is to generate Python code that creates random data for the given optimization model's sets and parameters. The dataset should be large, such that the optimization solver will take a few seconds to solve it.

1. Generate appropriate random data for all sets and parameters described in the model.

2. Follow these guidelines:
 - Use random and numpy for data generation
 - Include appropriate type hints
 - Add clear comments explaining the data generation logic
 - Ensure all required fields from the dataset class are populated
 - Make the data generation deterministic when a seed is provided
3. The datasets returned by the function should be small
4. Try to maintain simple relationships between the datapoints
5. All keys must be strings at all levels
6. When sampling random numerical values, make the range small relative to the value. For example, if the s
7. Include symmetry in the dataset. Some values should be the same intentionally.
8. Do not make the dataset sparse.
- 9.

Here is the description of the dataset
 {data_object_description}

Here is a template for the code:

```
import random
import numpy as np

def generate_data(seed=42):
    if seed is not None:
        random.seed(seed)
        np.random.seed(seed)

    data ={{}} # dictionary with strings as keys

    return data

# Format Instructions
{pydantic_format_instructions}

Problem Context
{user_input_problem_context}
```

The code should define a function that returns an instance of the dataset class defined in:
 {user_input_dataset_description}

Analysis Prompt

Used to analyze optimization models and identify improvement opportunities:

You are an expert in optimization modeling. You need to improve the efficiency of this optimization model. To achieve this, you need to propose several new sets of decision variables. For each new proposition, you should look at the implicit assumptions made by the given optimization model and check whether they actually need to hold or can be relaxed. Please list all assumptions and mention which ones could be relaxed and which ones should be kept.

Here are some propositions on how optimization models can be made more efficient by changing decision variables:

- aggregate variables representing groups of variables instead of individual instances in cases where decisions of individual instances are not important

- to can be derived in post-processing
- delete fixed variables
- replace redundant variables which can usually be detected through strict equalities between variables
- change semantics of variables by changing the definition of it such that the underlying problem can still be solved. Note that the application of pre- and post-processing is allowed, but not required
- integer relaxation may be used if the problem context logically guarantees that the optimal objective value does not change
- find an alternative problem structure
- removing an index from binary variables by indicating the level instead of individual assignments

{pydantic_format_instructions}

Analyze the following optimization case and provide a list of assumptions made in the model.

Here is a description of the optimization problem:

{user_input_problem_context}

Here is an associated optimization model in LaTeX format:

{canonical_formulation_latex}

A.2.2 Direct LLM Approach Prompts

Efficient Formulation Generation

Used in the direct LLM approach to generate more efficient formulations:

You are an expert in mathematical optimization. Given the following optimization problem and current formulation, create a more efficient formulation by changing the decision variables.

Problem Context:

{user_input_problem_context}

Current Formulation (LaTeX):

{canonical_formulation_latex}

Available Data (JSON):

Here is a description of keys available in the json file. The json file is loaded as an object called data. Access the data with data[key1] for 1-dimensional keys and data[key1][key2]... for multi-dimensional keys.

{user_input_dataset_description}

Your task is to:

1. Analyze the current formulation
2. Identify opportunities to change decision variables for better efficiency
3. Create a new formulation that is mathematically equivalent but more computationally efficient
4. Provide both LaTeX and GurobiPy code

Here are some propositions on how optimization models can be made more efficient by changing decision variables:

- aggregate variables representing groups of variables instead of individual instances in cases where decisions of individual instances are not important to can be derived in post-processing
- delete fixed variables

- replace redundant variables which can usually be detected through strict equalities between variables
- change semantics of variables by changing the definition of it such that the underlying problem can still be solved. Note that the application of pre- and post-processing is allowed, but not required
- integer relaxation may be used if the problem context logically guarantees that the optimal objective value does not change
- find an alternative problem structure
- removing an index from binary variables by indicating the level instead of individual assignments

A.2.3 Debugging and Error Correction Prompts

Data Generation Error Correction

Used to fix errors in data generation code:

Fix the following Python data generator code that encountered an error:

Error Information:

{error_info}

Original Data Generator Code:

{current_code}

Here is the data structure:

{user_inoput_data_description}

Please ensure the fixed code maintains the same data structure and logic while resolving the error.

GurobiPy Code Error Correction

Used to fix errors in GurobiPy model code:

Fix the following GurobiPy code that encountered an error:

Error Information:

{error_info}

Original GurobiPy Code:

{current_code}

Here is the data structure:

{user_input_data_description}

Please ensure the fixed code maintains the same model structure and logic while resolving the error.

A.3 World Food Program Case Study

This section provides a detailed case study of the World Food Program (WFP) emergency food distribution optimization problem, including the problem description, dataset structure, and both the original and reformulated mathematical formulations.

A.3.1 Problem Description

The World Food Program faces the critical challenge of optimizing emergency food distribution in crisis-affected regions such as conflict zones, post-disaster areas, or drought-stricken communities. The problem involves delivering

life-saving food aid to vulnerable populations while balancing cost efficiency, logistical feasibility, and nutritional adequacy.

Context and Challenges

Logistical Complexity:

- **Fragile Infrastructure:** Many regions lack reliable roads, with routes often disrupted by conflict, flooding, or landslides. Transshipment points (e.g., warehouses, temporary hubs) are critical but may have limited capacity.
- **Geographic Dispersion:** Beneficiary camps are frequently located in remote or inaccessible areas, requiring multi-stage transportation (supplier → transshipment point → camp).
- **Cost Variability:** Procurement costs for commodities vary by supplier, while transportation costs depend on distance, road conditions, and fuel availability.

Nutritional Needs:

- Each beneficiary requires a daily ration that meets minimum nutritional standards.
- Commodities differ in nutritional value; for example, rice provides carbohydrates but lacks certain micronutrients, necessitating a mix of foods.

Resource Constraints:

- Limited funding demands strict cost control across procurement and transportation.
- Political or security pressures may restrict access to certain suppliers or routes.

Key Objectives

The WFP aims to design a food distribution plan that:

- **Minimizes Total Costs:** Reduce expenses from procuring commodities and transporting them through complex networks.
- **Guarantees Food Availability:** Ensure all beneficiary camps receive sufficient quantities of each commodity.
- **Meets Nutritional Standards:** Provide rations that collectively satisfy all nutrient requirements.

Operational Components

- **Suppliers:** Costs vary by location and commodity.
- **Transshipment Points:** Intermediate hubs where food is consolidated, repackaged, or redirected. Bottlenecks must be avoided.
- **Beneficiary Camps:** Final destinations where displaced populations receive food. Demand is determined by the number of people and required ration sizes.

Critical Constraints

- **No Storage:** No storage is assumed—food is immediately redirected.
- **Demand Fulfillment:** Each camp must receive enough of every commodity to meet its calculated ration size.
- **Nutritional Adequacy:** The combination of commodities in the ration must meet all nutrient thresholds.

A.3.2 Dataset Structure

The WFP dataset contains the following components:

Sets

- **nutrients**: List of nutritional categories
- **foods**: List of food commodities
- **beneficiary_nodes**: List of beneficiary camps
- **supplier_nodes**: List of suppliers
- **transshipment_nodes**: List of transshipment points
- **all_nodes**: Combined list of all node types (suppliers + transshipment + beneficiaries)
- **edges**: Adjacency matrix representation of edges between nodes
- **paths**: List of possible paths from suppliers to beneficiaries, including direct routes and routes with up to 2 transshipment nodes

Parameters

- **demand**: Number of beneficiaries at each camp
- **nutritional_requirements**: Required amount of each nutrient per person
- **nutritional_values**: Nutritional content of each food item per kg
- **path_costs**: Cost of transporting food along each path
- **path_end_indicators**: Binary indicators for valid path endpoints
- **procurement_costs**: Procurement cost per food item
- **edge_costs**: Cost of transporting food along each edge

A.3.3 Inefficient Formulation

The original formulation uses a path-based approach with the following mathematical structure:

Sets

- P : Set of all simple paths from suppliers to beneficiary camps.
- K : Set of commodities.
- L : Set of nutrients.
- N_B : Set of beneficiary camps.

Parameters

- c_{pk} : Cost of shipping one kg of commodity $k \in K$ along path $p \in P$.
- q_k : Procurement cost per kg of commodity $k \in K$.
- $\text{nutval}_{k\ell}$: Nutrient- ℓ content (per kg) of commodity $k \in K$.
- nutreq_{ℓ} : Per-person requirement for nutrient $\ell \in L$.
- dem_j : Number of beneficiaries at camp $j \in N_B$.
- $e_{jp} = \begin{cases} 1, & \text{if path } p \text{ ends at camp } j, \\ 0, & \text{otherwise} \end{cases} \quad \text{for all } j \in N_B, p \in P.$

Variables

- $x_{pk} \geq 0$: Amount (kg) of commodity $k \in K$ shipped along path $p \in P$.
- $R_k \geq 0$: Ration size (kg per person) of commodity $k \in K$.

Mathematical Formulation

$$\begin{aligned}
\min \quad & \sum_{p \in P} \sum_{k \in K} c_{pk} x_{pk} + \sum_{k \in K} q_k \cdot \left(\sum_{p \in P} x_{pk} \right) \\
& \text{Minimize total cost of procurement and transportation} \\
\text{s.t.} \quad & \sum_{p \in P} e_{jp} x_{pk} \geq \text{dem}_j \cdot R_k, \quad \forall j \in N_B, \forall k \in K \\
& \text{Ensure each camp receives required ration quantities} \\
& \sum_{k \in K} \text{nutval}_{kl} \cdot R_k \geq \text{nutreq}_l, \quad \forall l \in L \\
& \text{Ensure minimum nutritional content per person} \\
& x_{pk} \geq 0, \quad \forall p \in P, \forall k \in K \\
& R_k \geq 0, \quad \forall k \in K \\
& \text{Define variable domains}
\end{aligned}$$

A.3.4 Efficient Formulation

The reformulated model uses an arc-based multi-commodity flow approach, which provides better computational efficiency:

Sets

- N : Set of nodes.
- N_S : Set of suppliers ($N_S \subseteq N$).
- N_T : Set of transshipment points ($N_T \subseteq N$).
- N_B : Set of beneficiary camps ($N_B \subseteq N$).
- E : An N by N matrix with 0-1 indication that an edge exists.
- L : Set of nutrients.
- K : Set of commodities.

Parameters

- dem_i : Number of beneficiaries at node $i \in N_B$.
- pc_k : Procurement cost of commodity $k \in K$ (\$/kg).
- tc_{ijk} : Transportation cost of commodity $k \in K$ from node $i \in N$ to node $j \in N$ (\$/kg).
- nutreq_l : Nutritional requirement of one beneficiary for nutrient $l \in L$.
- nutval_{kl} : Nutritional value of commodity $k \in K$ for nutrient $l \in L$ (per kg).

Variables

- F_{ijk} : Amount of commodity $k \in K$ sent from node $i \in N$ to node $j \in N$ (kg).
- R_k : Ration size of commodity $k \in K$ in one person's portion (kg).

Mathematical Formulation

$$\min \sum_{k \in K} \text{pc}_k \left(\sum_{j \in N_B} \text{dem}_j R_k \right) + \sum_{i \in N} \sum_{j \in N} \sum_{k \in K} \text{tc}_{ijk} F_{ijk}$$

Minimize total procurement and transportation costs

$$\text{s.t.} \quad \sum_{i \in N} E_{ij} F_{ijk} = \sum_{i \in N} E_{ji} F_{jik}, \quad \forall j \in N, \forall k \in K$$

Flow preservation at transshipment points for each food

$$\sum_{i \in N} E_{ij} F_{ijk} \geq \text{dem}_j R_k, \quad \forall j \in N_B, \forall k \in K$$

Beneficiary camps receive demanded food quantities

$$\sum_{k \in K} \text{nutval}_{kl} R_k \geq \text{nutreq}_l, \quad \forall l \in L$$

Meet nutritional requirements for all nutrients

$$R_k, F_{ijk} \geq 0, \quad \forall i, j \in N, \forall k \in K$$

Non-negativity constraints